Digital Vector · VCE Applied Computing | Unit 2 · Area of Study 1 · Innovative Solutions · KK08

One great idea is the last idea you keep — not the first you have.

Meet Team Vector — Jade, Marcus, Priya and Tom, four Year 11 students at a Melbourne school. They've spotted an opportunity : new Year 7 students arrive lost, anxious, and afraid to ask for help in their first weeks. Team Vector decides to develop an innovative digital solution. The question is how — and "we'll just th

What this key knowledge covers
This summary distils everything you need for **KK08 — One great idea is the last idea you keep — not the first you have.** in Innovative Solutions. Work through the explanations, then use the worked good-vs-weak answers to see exactly how marks are won and lost. Tick off the revision checklist at the end before a SAC or exam.

Study summary

Developing a solution is a process, not a moment

Meet **Team Vector** — Jade, Marcus, Priya and Tom, four Year 11 students at a Melbourne school. They've spotted an *opportunity*: new Year 7 students arrive lost, anxious, and afraid to ask for help in their first weeks. Team Vector decides to develop an innovative digital solution. The question is **how** — and "we'll just think of something good" is not a method.

VCAA describes the thinking behind this stage precisely. **Design thinking** is "a way of thinking critically and creatively to generate and evaluate innovative ideas, and precisely define the preferred solution." It runs on two engines you must be able to name and tell apart:

Divergent thinking opens the problem up — it generates a wide *range* of ideas (quantity, variety, creativity). **Convergent thinking** closes it down — it evaluates options and *selects the preferred idea* to develop. You diverge first, then converge. Mixing them up — or doing only one — is the classic mistake.

Get to know the end user — their needs, frustrations and context. Not order-taking.

Brainstorm, mind-map and ideate across perspectives to generate many ideas.

Cluster, evaluate against criteria, and select the preferred idea.

Flesh the chosen idea into a workable proof-of-concept or prototype.

A director doesn't hire the first actor through the door. They **diverge** — audition dozens, encouraging wild range — then **converge**, comparing every audition against the role's real

requirements until one stands out. Developing a solution works the same way: many auditions, one cast lead, then weeks of rehearsal to develop the performance.

Get inside the user's head before you touch an idea

Empathising means deliberately understanding the end user — what they say, think, do and feel — so your ideas solve their *real* problem, not the one you assumed. Team Vector interviews **Sam**, a quiet Year 7 who started mid-year and got lost three times in week one.

Build an empathy map

An **empathy map** sorts what you learn into four quadrants. It's the most exam-friendly empathise tool because it forces you past surface wants. Drag Sam's interview snippets into the right quadrant.

The same problem looks different through every set of eyes

"Ideating across perspectives" means deliberately viewing the problem from each *stakeholder's* point of view before you brainstorm. The new-student problem isn't only Sam's — it touches buddies, teachers and parents, and each lens unlocks different ideas.

Go wide: generate far more ideas than you'll ever use

Divergent thinking is about **quantity and range**. The single rule that makes it work: *defer judgement.* No "that's silly," no "we can't afford that" — yet. Criticism in the diverge phase kills the wild ideas that often become the best ones.

Brainstorming

Rapid-fire ideas out loud or on sticky notes. Build on others' ideas ("yes, and…"). Aim for volume.

Mind mapping

Start with the problem in the centre; branch outward into themes, then sub-ideas. Shows connections.

SCAMPER & "How might we…"

Prompts that force new angles — Substitute, Combine, Adapt… Each prompt is a fresh idea engine.

Team Vector runs a ten-minute brainstorm and fills the wall. Some ideas are obvious, some are wild, a few are terrible — and that's exactly right. Flip each note to see the idea.

A fisher who drops one line catches one fish. Divergent thinking casts a wide net so there's something worth keeping when you haul it in. You throw most of it back — but you can't keep what you never caught.

Go narrow: evaluate, then choose with reasons

Now flip the engine. Convergent thinking **clusters** related ideas, evaluates them against *criteria*, and selects a **preferred idea**. The discipline here is choosing on evidence — feasibility, user fit, originality — not on whose idea it was.

Left edge = full divergence: ideas explode outward, many and varied. Slide right and convergent thinking pulls them together, dimming the weak ones until a single preferred idea glows at the funnel's neck. That neck is the moment you commit.

Convergent tools: how the choice gets made

Group similar sticky notes into themes so 30 ideas become 5 directions.

Each team member spends a few dots on the directions they back. Fast, democratic shortlist.

Rate each shortlisted idea against the criteria (feasibility, user fit, originality). Highest total wins — and you can *justify* it.

Tell the two engines apart

This is the single most-tested distinction in this dotpoint. Sort each technique by the thinking it uses.

Turn the chosen concept into something real

Picking the preferred idea isn't the finish line — it's the start of **development**. Now Team Vector adds detail, resolves the unknowns, and builds a *proof of concept* so they can test whether Compass actually helps Sam.

From idea to proof-of-concept

Developing means making the abstract concrete: defining each feature, sketching screens, deciding how a buddy gets paired, and building enough of it to demonstrate the core promise. VCAA accepts the developed solution as a **proof of concept, prototype or product**.

Test the risky assumption first

The riskiest assumption in Compass: will a shy Year 7 actually message an older buddy? Team Vector builds that one flow first and tests it with Sam — because if that fails, nothing else matters. Develop what's uncertain, not what's easy.

VCE-style questions with weak vs strong answers

Read each question, draft your answer, then reveal a weak attempt, a strong one, and the marking guide. Self-mark honestly — your total feeds the dock.

Developing solutions — everything in one place

- **Empathise** — know the user (says/thinks/does/feels)
- **Ideate** across perspectives — many lenses, many ideas
- **Diverge** — generate a wide range, defer judgement
- **Converge** — evaluate vs criteria, select the preferred idea
- **Develop** — build a tested proof of concept
- **Divergent** = go wide, *generate* a range
- **Convergent** = go narrow, *select* the preferred idea
- Diverge **then** converge — never mix them
- Brainstorming · mind mapping
- SCAMPER · "How might we…"

- Rule: **defer judgement**
- Affinity clustering · dot voting
- Scoring matrix against **criteria**
- Choose with reasons, not preference
- Tie every technique to its **job + stage**
- Develop = whole arc, not just "build it"
- Justify the preferred idea against criteria
- range / variety · defer judgement
- evaluate · criteria · preferred idea
- user-centred · proof of concept

Common traps & misconceptions

Exam trap
× TRAP 1 Swapping divergent and convergent Writing that brainstorming "narrows down ideas," or that a scoring matrix "generates ideas." They're opposites. **Fix:** Anchor it: *divergent = generate a range* (go wide), *convergent = select the preferred* (go narrow). Diverge then converge.

Exam trap
× TRAP 2 Treating "develop" as only "build the app" Jumping straight to coding skips empathise, ideate and converge — the marks live in the *process*. **Fix:** "Developing a solution" covers the whole arc from understanding the user to a tested proof-of-concept, not just the final build.

Exam trap
× TRAP 3 Judging ideas during brainstorming Saying you "picked the realistic ideas while brainstorming" breaks the one rule of divergence. **Fix:** Name the rule: *defer judgement.* Evaluation belongs to the convergent stage, not the divergent one.

Exam trap
× TRAP 4 Confusing empathise with order-taking "We asked the user what features they wanted" is requirements-gathering, not empathy. **Fix:** Empathising uncovers needs, context and *feelings* (what they think/do/feel), often things the user can't articulate as a feature.

Exam trap
× TRAP 5 Picking the preferred idea "because we liked it" A choice with no criteria can't be justified — and *justify* questions are common here. **Fix:** Select against stated criteria (feasibility, user fit, originality, scope) so you can give reasons. That's what convergent thinking is for.

Exam trap
× TRAP 6 Listing techniques with no purpose "Brainstorming, mind mapping, dot voting" — a bare list earns few marks if you don't say what each *does*. **Fix:** Tie each technique to its job and stage: mind mapping (divergent — generate & connect ideas), dot voting (convergent — shortlist the preferred).

How to answer exam questions

Read the **command word** first — it sets how much to write. *State/Identify* wants a word or phrase; *Describe* wants features; *Explain* wants reasons and links; *Justify/Evaluate* wants a judgement backed by evidence. Always pull in the **scenario detail** rather than answering in the abstract. The examples below contrast a weak response with a strong one for the same question.

Q1. Distinguish · 2 marks

Distinguish between divergent thinking and convergent thinking when developing an innovative solution.

✗ A weak answer
— 0–1 marks Divergent thinking is when you think of ideas and convergent thinking is when you think harder about them. Vague and almost circular. "Think harder" isn't a distinction. No mention of generating a *range* vs *selecting* the preferred idea.

✓ A strong answer
— 2 marks Divergent thinking generates a wide range of varied ideas for the solution (e.g. brainstorming many options, deferring judgement), whereas convergent thinking evaluates those ideas against criteria to select a single preferred idea to develop. Both sides defined by their job — generate vs select — with a clear contrast word. Examples make it concrete.

How the marks are awarded
- **1 mark** — divergent = generating a range/variety of ideas.
- **1 mark** — convergent = evaluating/selecting the preferred idea.

Q2. Describe · 4 marks

Describe two techniques Team Vector could use to generate a range of ideas, and explain why generating across multiple perspectives strengthens their solution.

✗ A weak answer
— 1–2 marks They could use brainstorming and mind mapping. Perspectives are good because more people means more ideas and that is better for the team. Names two techniques but doesn't *describe* them. The perspectives reason is hand-wavy ("more is better") and misses user-centred design.

✓ A strong answer
— 4 marks Brainstorming — the team rapidly calls out as many ideas as possible without judging them, building on each other to maximise volume. Mind mapping — they place the new-student problem at the centre and branch outward into themes (wayfinding, social, safety), revealing connections and gaps. Ideating across perspectives (Sam, a buddy, a teacher, IT) strengthens the solution because each lens surfaces different needs and constraints — e.g. a teacher needs it not to distract class, IT needs it to run on the school network — leading to a more *user-centred*, robust preferred idea. Two techniques genuinely described (what + how), plus a perspectives reason tied to real stakeholder needs and user-centred design.

How the marks are awarded

- **1 mark each** — two divergent techniques described, not just named (max 2).
- **1 mark** — perspectives surface differing stakeholder needs/constraints.
- **1 mark** — links to a stronger, user-centred solution.

Q3. Explain · 6 marks

Team Vector has selected Compass as its preferred idea. **Explain** what "developing the preferred idea" involves, and **justify** which feature they should develop and test first.

✗ A weak answer

— 2 marks Developing the idea means making the app. They should develop the home screen first because every app needs a home screen and it is easy to do. "Making the app" is shallow. Choosing the easiest feature is backwards — and there's no link to risk or to the user's real need.

✓ A strong answer

— 6 marks Developing the preferred idea means turning the chosen concept into something concrete and testable — defining each feature, designing the screens, and building a proof of concept that demonstrates Compass's core promise, rather than just coding randomly. They should develop and test the buddy-message flow first, because it carries the riskiest assumption : that a shy student like Sam will actually reach out to an older buddy. If that assumption fails, the whole concept fails, so testing it early is the highest-value development. It also loops straight back to the empathy insight — Sam feels watched — so building a private, discreet version of that flow tests both feasibility and user fit at once. Explains development as conceptconcretetetested proof-of-concept, then justifies the first feature by risk and by traceability back to the empathy stage.

How the marks are awarded

- **1–2 marks** — developing = making the concept concrete and testable (proof of concept / prototype), defining features & designs.
- **1 mark** — identifies a sensible first feature.
- **1–2 marks** — justifies it by risk / riskiest assumption (test the uncertain thing first).
- **1 mark** — links the choice back to the user need / empathy insight.

Q4. Practice question

Scenario: Sam says "I just want a map." In their empathy work, Team Vector notices Sam looks anxious and avoids asking older students for directions.

✗ A weak answer
— 1 mark Empathising is good because you understand the user better, so you can make a better app that they will like more. Generic. Never uses Sam's scenario, never names the deeper need, and doesn't explain the *link* to a better preferred idea.

✓ A strong answer
— 3 marks Asking Sam what app to build only captures a surface want (a map). Empathising reveals the deeper need — Sam feels anxious and embarrassed asking older students for help, which Sam never states directly. This reframes the brief, so the preferred idea can target confidence (private, discreet directions), not just navigation — making it more likely to actually solve Sam's real problem. Surface want vs real need reframed brief better-targeted preferred idea. Three clean links, all grounded in the scenario.

How the marks are awarded

- **1 mark** — identifies that asking directly gives only a stated/surface want.
- **1 mark** — empathising surfaces an unstated need/feeling (anxiety, embarrassment) using the scenario.
- **1 mark** — links this to a better-targeted preferred idea.